Vision-Guided Autonomous Chess with State Validation and Recovery on a UR5 Collaborative Robot

Luis A. Gonzalez-Guevara
Faculty of Engineering
Universidad de La Sabana
Chía, Colombia
ORCID: 0009-0003-5224-359X

Juan Sebastian Ulloa Mejia
Faculty of Engineering
Universidad de La Sabana
Chía, Colombia
ORCID: 0009-0008-7405-4251

Jorge Alberto Castellanos Rivillas
Faculty of Engineering
Universidad de La Sabana
Chía, Colombia
jorge.castellanos@unisabana.edu.co

David Celeita
Faculty of Engineering
Universidad de La Sabana
Chía, Colombia
ORCID: 0000-0002-8849-1777

***Abstract*—This paper presents a vision-guided autonomous chess system built around a UR5 collaborative robot. A wrist-mounted camera, homography-based board rectification, and two lightweight convolutional neural networks estimate square occupancy and piece color. Board-state transitions are validated against chess rules before robot execution, while a supervisory controller integrates Stockfish reasoning, Cartesian coordinate conversion, URScript motion, turn synchronization, and recovery from illegal or missing human moves. The vision pipeline achieved 98.25% occupancy accuracy and 97.12% color-classification accuracy under variable illumination, with end-to-end move times of 9–18 s. The system demonstrates reliable integration of robot perception, symbolic reasoning, collaborative manipulation, and fault-aware human–robot interaction.**

***Index Terms*—autonomous robots, robot perception, vision-guided manipulation, human–robot interaction, collaborative robots**

I. Introduction

Advances in robotics have increasingly addressed human–robot interaction (HRI) in entertainment and game-based applications [1, pp. 1, 21–22]. The integration of artificial intelligence (AI) and robotics further enables autonomous perception, decision-making, and physical interaction, making advanced robotic technologies more accessible to non-specialist users [1, pp. 21–22].

Autonomous chess-playing systems have progressively evolved from restricted experimental environments toward real-world settings [2, p. 4291]. Computer vision has played a central role in this transition: It enables automatic chessboard localization and state estimation [3, pp. 3–4]. Current systems combine geometric board detection with learning-based methods for chess-piece detection and classification [4, p. 1] [5, pp. 1, 4], which improve robustness under practical variations [6, p. 1] [3, pp. 2–3].

Under educational and interactive contexts, collaborative manipulators have become common due their integrated safety features, and programming accessibility [7, p. 5] [8, p. 5]. Their combination with perception and decision-making systems enables physical interaction with objects in a shared human–robot workspace.

This paper presents a modular architecture for an autonomous chess-playing system based on a CNN-driven vision engine and a UR5 collaborative robot. The architecture separates perception, chess reasoning, supervisory control, robot manipulation, and user interaction. The vision pipeline combines OpenCV-based chessboard localization with two cascaded CNN classifiers for square occupancy and piece-color estimation. The resulting system provides an end-to-end integration of computer vision, symbolic reasoning, robotic manipulation, and fault-aware human–robot interaction.

II. Case Study

Chess constitutes a constrained testing environment for embodied human–robot interaction known as HRI, where a person and a robot share a physical workspace, alternate turns, interpret a common task state, and act under explicit rules [1, pp. 1–2, 22] [9]. Furthermore, physical game distinguishes this setting from screen-based digital play and can influence how people experience robotic game partners [10, 11].

Interaction requires a visual-perception pipeline and a fail-safe supervisory layer. The visual pipeline must infer the human player's move from camera observations of a standard chessboard while remaining tolerant to changes in board pose, perspective, and illumination [6, pp. 1–2]. The system must use a chess engine to verify the perceived position and the

This work was supported by the project "Synergizing Data and Simulation: Digital Twins and Cloud Environments for Acceleration and Optimization of Energy Transition (SDS-DTC4ET ID ING-86-20251)" at the Faculty of Engineering, Universidad de La Sabana, Colombia with the Project ID ING-86-2025. It is product of the efforts from researcher members from "Semillero de Robotica Aplicada" of Universidad de La Sabana. We are entirely thankfull by their efforts as support.

inferred move constitutes a legal transition from the previous game state [2, p. 4293]. The supervisory layer withholds physical execution and initiates recovery whenever an invalid or inconsistent state is detected [1, p. 8], which plays an important role on HRI context [12].

Geometric board localization and homography-based rectification normalize the camera view, following established planar-calibration and corner-localization methods [13, 14]. Two square-level CNN classifiers estimate occupancy and piece color. The inferred transition is then checked against the previous FEN state and the rules of chess; only validated moves are converted through the move engine into UR5 pick-and-place commands [5, pp. 1–2, 4–5]; [7, pp. 7–8, 11–12]. This separation of perception, game reasoning, and robot control is consistent with related chess and checkers architectures [1, 15].

The chessboard as the shared physical workspace and as a visible representation of the game state. A turn-shift button coordinates turn handovers, while the GUI supports game configuration, displays the perceived position, and provides feedback to the player. An accessible emergency-stop mechanism preserves human oversight. In addition, state-validation and recovery routines detect illegal or inconsistent human moves before the robot generates its response. When an illegal displacement is identified, the system prevents the invalid transition from updating the internal chess state and commands the UR5 to return the misplaced piece to its previous valid square [16, 17].

III. Hardware and Software Settings

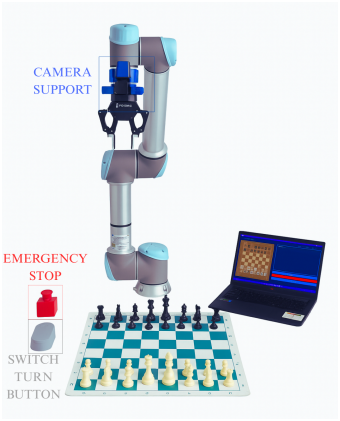


Fig. 1. Hardware setting for the autonomous chess playing UR5 robot.

System uses the Hardware and Software configuration denoted in Figure 1 and Table I. The emergency-stop device remains accessible to the player and provides hardware-level interruption of robot operation.

IV. System Architecture

The software architecture is organized into five principal modules: the Main Loop Controller (Section IV-A), Vision Engine (Section IV-B), Chess Reasoner Engine (Section IV-C), Move Engine (Section IV-D), and Graphical User Interface (Section IV-E). Each module follows object-oriented design principles, enabling functional separation, independent debugging, and future extension of the system.

TABLE I
Hardware and Software Settings

Item	Setting
Robot	UR5 collaborative robot
Gripper	Robotiq; TCP port 63352
Camera	Wrist-mounted RGB camera; 1920 × 1080 pixels; nadir configuration
Board and safety	56.5-mm squares; turn-shift button connected to UR5 digital I/O; emergency stop
Python	3.13.9
Vision and ML	OpenCV, TensorFlow/Keras, NumPy; library versions not reported
Chess and GUI	Stockfish via UCI, `python-chess`, Tkinter
Robot communication	URScript over TCP port 30002; robot-state feedback over port 30003

A. Main Loop Controller

The main loop Controller coordinates the system through three principal stages: initialization, turn synchronization, and turn execution with state control.

1) Initialization: The routine `Calibracion()` uses the function `cv2.findChessboardCornersSB` to detect the board pattern. The pixel displacement between the image center and the board center is used to reposition the manipulator above the board.`cv2.findChessboardCornersSB` is executed again to obtain the 7×7 grid of inner corners. The method `cv2.findHomography` then estimates the matrix $\mathbf{H}$ that maps ideal board coordinates (X, Y) to image coordinates (u, v):

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}. \tag{1}$$

The homography is estimated using RANSAC to reduce the influence of incorrect corner detections:

$$\mathbf{H} = \arg\min_{\mathbf{H}} \sum_{i=1}^{49} \rho\left(\|\mathbf{p}_i - \mathbf{H}\mathbf{q}_i\|^2\right). \tag{2}$$

The resulting homography is used to extrapolate a stable 9×9 grid shwon in 2, which includes the outer board boundaries. Square labels are assigned using the global mapping constants `COLUMNAS_TABLERO = "hgfedcba"` and `FILAS_TABLERO = "12345678"`. It gives consistent algebraic notation between the perception and motion modules.

An external board quadrilateral can be used to estimate an equivalent homography. Nevertheless, detection from the inner corner pattern is preferred: it preserves the expected geometric spacing of the board.

Finally, the GUI allows the player to select their color and the chess engine difficulty. After loading these parameters and



Fig. 2. Original camera image and homography-rectified chessboard.

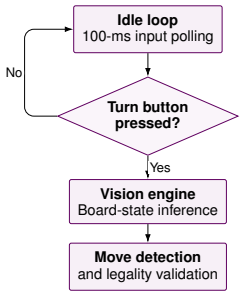


Fig. 3. Turn synchronization and activation of the perception and move-validation pipeline.

establishing the UR5 connection, the controller enters the turn-synchronization state described in Section IV-A2.

2) Turn Synchronization: After initialization, the controller enters an idle loop and reads the turn-shift button every 100 ms. It follows the logic behind Figure 3. Pressing the button indicates that the human player has completed their turn.

3) Turn Execution and State Control: Once the turn-shift signal is detected, the inferred board transition is evaluated against the current chess state using the `python-chess` library. The controller then executes a valid move, recovers from an illegal board transition, or warns the player when no human move is detected.

For a legal human move, the controller updates the move history and requests a response from Stockfish. The selected move is converted from UCI notation into robot coordinates, while captures, castling, en passant, and promotion are handled before the corresponding URScript and gripper commands are executed by the UR5.

If the detected board transition is illegal, the robot returns the misplaced piece to its previous square and the GUI warns the player. If the button is pressed without a detectable board change, the GUI indicates that the player must complete a move. These branches converge in the same synchronization step, in which the interface is updated and the UR5 presses the turn-shift button.

At the end of the cycle, the controller evaluates the internal chess state for a terminal condition. If the game continues, the system returns to the idle loop shown in Fig. 3. Otherwise, the final result is displayed and the game-setup procedure is restarted.

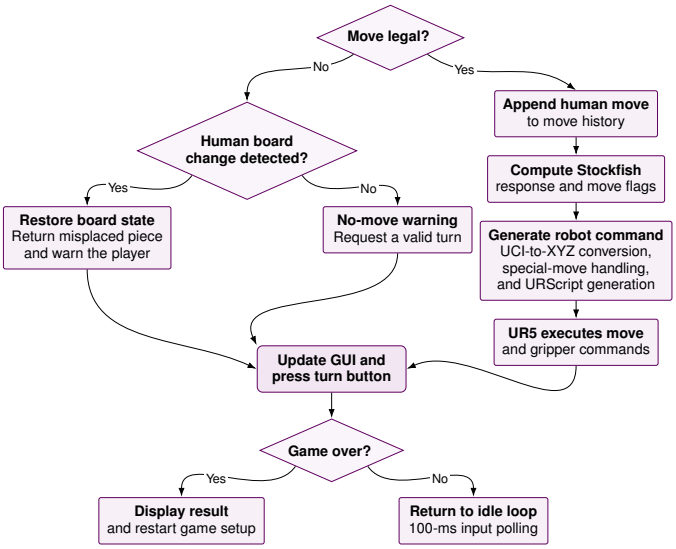


Fig. 4. Turn execution, invalid-state recovery, synchronization, and terminal-state handling.

B. Vision engine

Vision Engine ensures piece detection and classification. It operates at three fully automatic different stages: Game initialization, per-square classification stage, and a post-processing which symbolical represents the state of the chessboard into FEN. The first stage was already described in the section IV-A1, in which from $\mathbf{g9}$, each square r, c is defined by its four projected corner points. Then, a per-cell homograph $\mathbf{M}_{rc}$ warps the square to a normalized 90×90 RGB patch using `cv2.getPerspectiveTransform` and `cv2.warpPerspective`. It is cut down into 64 patches and normalized their RGB values to $[0, 1]$.

Two lightweight CNNs in cascade perform the classification tasks for each patch, as shown in Figure 5. Thresholds are maintained near to 0.72, the ROC and threshold analysis is done on Figure 6. This process outputs an 8×8 boolean occupancy grid and an 8×8 integer color grid with labels 0 for empty, 1 for white, and 2 for black. Datasets characteristics and augmentations are shown in 5, also training hyperparameter and training splits.

Finally, the booleans outputted by the second stage are transformed into FEN nomenclature. The FEN and board matrix representations are injected into the Chess Reasoner Engine, discussed in IV-C, for chess motor to process the current board as an historial of previous movements.

C. Chess reasoner engine

The Chess Engine computes the best move by driving Stockfish through a persistent UCI session. At the start of each game, it initializes the engine with the player's color and a target Elo, via `UCI_LimitStrength`, and `UCI_Elo` and sets a fixed thinking time per move. Given the move history, it reconstructs the position, queries the best move, validates the legality of the move against the current board using the `Chess`

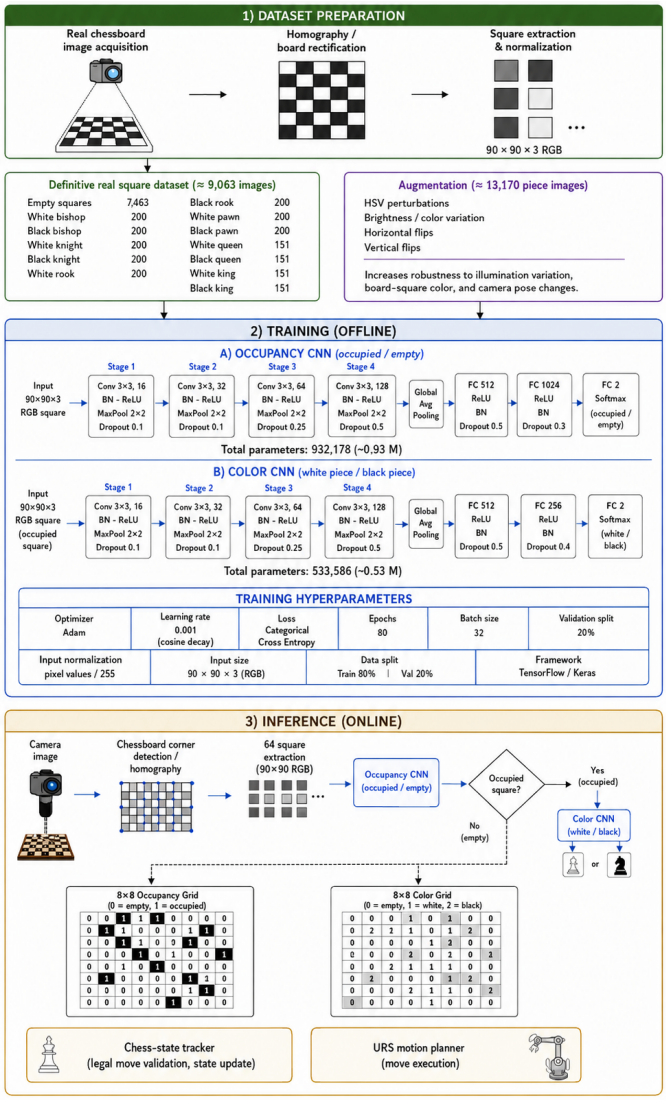


Fig. 5. Vision-engine pipeline showing dataset acquisition, offline CNN training, and online board-state inference.

library. The module explicitly flags captures and castling for downstream logic and reports terminal outcomes.

D. Move engine

The module bridges CNN-based perception and chess logic with direct robot actuation. The engine communicates with the UR controller via sockets. Motion commands are injected as URScript programs through port `30002`, while the real-time interface on port `30003` is polled to monitor the TCP pose and joint states. The Robotiq gripper is controlled over a dedicated ASCII channel on port `63352`. This arrangement allows the system to issue new motion commands while verifying the robot state before proceeding.

Each motion is executed by sending a URScript snippet that defines a TCP pose as the Equation 3.

$$\mathbf{p} = [x, y, z, r_x, r_y, r_z] \tag{3}$$

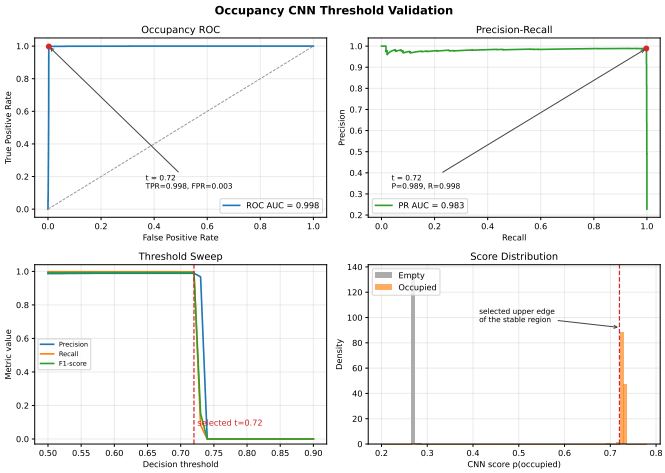


Fig. 6. Threshold empirical analysis for the detection and classification threshold values used on the pipeline.

where translational coordinates are expressed in meters and rotational coordinates in radians.

At execution time, the controller computes inverse kinematics using `get_inverse_kin` and performs a joint-space `movej`. The script is transmitted via port `30002`, while in parallel the engine polls port `30003` to obtain the current TCP pose and joint configuration. The function `move_to` enforces a positional tolerance before confirming that the robot reached its target.

The engine performs automatic calibration followed by systematic square-coordinate generation. The routine `Calibracion()` is described in Section IV-A1. The yaw is compensated by rotating the sixth joint with a `movej` command, and the pixel offset is converted to millimeters using the known square size of 56.5 mm. Combined with the current TCP pose from port `30003`, this yields a planar homogeneous transform $\mathbf{T}$, which is shown on the equation 4, which maps local board coordinates to the robot base.

If automatic detection fails, the fallback `_calibracion_manual()` estimates yaw from two known board points (p_{h8} and p_{a8}). With $\mathbf{T}$ fixed, `_generar_casillas()` computes positions for all 64 squares on an 8×8 grid and assigns two poses per square: a hover height z_{sobre} and a grasp/place height z_{agarre}, stored in dictionaries for constant-time lookup.

For grasping and placing commands, each pick-and-place pose is computed from the board-to-robot calibration and solved through the UR5 inverse kinematics. The robot approaches the square vertically, descending from the configured pre-grasp height $z_{\text{sobre}} = 115\,\text{mm}$ to the grasp height $z_{\text{agarre}} = -16\,\text{mm}$, while the Robotiq gripper uses position commands of 100 to open and 250 to close, with speed and force set to 150 and 250, respectively. After grasping a piece, the robot retracts, transfers it to the target square, releases it, and clears the board.

$$\mathbf{T} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & x \\ \sin\theta & \cos\theta & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4}$$

Where $\theta = \arctan 2(p_{a8,y} - p_{h8,y}, p_{a8,x} - p_{h8,x})$ is the rotational angle given by the known position of A8 and H8 by `Calibracion()`

Built on top of `move_to`, higher-level routines—`pick`, `place`, `move_pieza`, and `take`—compose approach, descent, gripper actuation, and retreat into reusable patterns. A complete chess move can be invoked from the main program with a single call such as `move_pieza("e2","e4")`. This API hides calibration, transforms, inverse-kinematics validation, and sequencing, reducing software complexity and improving robustness during gameplay.

E. Graphical User Interface -GUI-

The GUI provides a user-friendly dashboard for the system's core functions, including controls to select the player's color and engine difficulty at the start of each game. Difficulty presets map to an Elo-like range of 800–2500 and a thinking time of 300–3000 ms per move; these settings are passed to the chess engine in IV-C to align playing strength and search depth with the selected level, which accommodates both beginners and advanced players. A live match image updates after each human or robot action, while the `Unicode` board view lets users follow the position.

V. System Validation and Experimentation

The complete system was deployed in 70 physical chess matches: 50 matches were conducted indoors under laboratory conditions and 20 matches were conducted outdoors. The outdoor deployment exposed the perception system to greater variations in ambient illumination. Each match exercised the complete perception–reasoning–action pipeline.

During valid gameplay, the manipulation layer supported standard chess moves as well as captures, castling, en passant, and promotion. Before any robot response was executed, the observed human-board transition was validated against the rules associated with the current internal chess state.

TABLE II
Stage-wise execution-time statistics for a standard chess move.

Stage	n	Mean (s)	SD (s)	Main source of variability
Vision CNN	40	1.872	1.074	CPU-only inference and TensorFlow thread contention.
Stockfish	80	1.003	0.002	Fixed 1-s search budget; residual UCI communication latency.
UR5 motion*	4032	12.951	0.755	Joint-space displacement produced by inverse kinematics.
Complete cycle	–	**15.826**	**1.313**	Combined variability of the three sequential stages.

*The UR5 motion time was chronometer from the user. CNN and Stockfish times were measured directly using the production software.

A. Vision Pipeline performance

The dual-CNN architecture achieved 98.25% occupancy detection and 97.12% color classification accuracy respectively as Figure 7 shows.

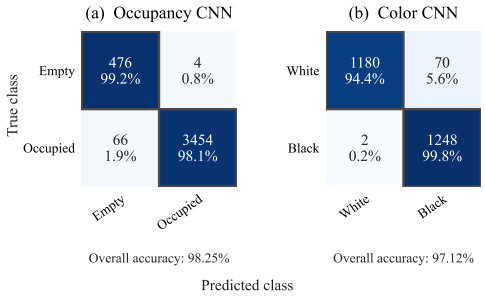


Fig. 7. Confusion matrices for the occupancy and color CNNs.

The luminance-adaptive data synthesis worked on different luminance environments, where false negatives occurred under extreme illumination scenarios. In these cases, the system's validation pipeline correctly detected illegal board states and prompted human intervention.

B. Robot Execution Performance

Average movement execution time is recorded on Table II, which demonstrates comfortable time execution for HRI. The automatic calibration routine successfully recovered board pose and orientation via `findChessboardCornersSB`, with the remaining cases resolved through the fallback external-quadrilateral detection method. The fixed 9×9 homography grid demonstrated stability across game sessions, eliminating the need for per-frame recalibration.

C. Fault Tolerance and Recovery

State validation is performed before a perceived human move is accepted by the internal game state or used to generate the robot's response. The observed board transition is compared with the legal moves available from the current chess position. The recovery logic addresses several categories of invalid human actions, including moving a piece to an illegal destination, attempting castling when the required chess conditions are not satisfied, attempting en passant when the move is unavailable, and executing a move that does not resolve an active check. A missing-move condition is also detected when the turn-shift button is pressed without a valid board transition.

When an illegal piece displacement is detected, the invalid transition is not committed to the internal chess state. Instead, the supervisory controller determines the previous valid square and commands the UR5 to physically return the misplaced piece to its expected position. The GUI simultaneously informs the player that the attempted move was invalid. After restoration, the system retains the previous valid FEN state and resumes the turn-synchronization procedure.

Robot motion is monitored through the UR5 real-time interface on port `30003`. The `move_to` routine verifies

that the controller-reported TCP position lies within a 2-mm acceptance tolerance of the commanded target before the manipulation sequence proceeds. This threshold is used as an internal controller-based completion criterion. An accessible emergency-stop device provides independent hardware-level mechanism for interrupting robot operation.

VI. Conclusion

This paper presented a modular CNN-based autonomous chess system for human-robot interaction on the UR5 platform. The architecture separates the system into five principal software modules, enabling independent development, debugging, and future extension of the perception, reasoning, manipulation, supervisory-control, and user-interface components.

The dual-CNN vision pipeline achieved 98.25% occupancy and 97.12% color classification accuracy under variable illumination, while the automatic calibration and fault-aware control mechanisms were deployed across 70 physical chess matches, including 50 indoor laboratory matches and 20 outdoor matches. Average move execution time of 9 to 18 seconds proved adequate for comfortable human-robot interaction, while the modular design supported adaptation and extension for educational purposes.

References

[1] R. Zhang, J. de Winter, D. Dodou, H. Seyffert, and Y. B. Eisma, "An open-source reproducible chess robot for human-robot interaction research," *Frontiers in Robotics and AI*, vol. 12, Apr. 2025. [Online]. Available: http://dx.doi.org/10.3389/frobt.2025.1436674

[2] C. Matuszek, B. Mayton, R. Aimi, M. P. Deisenroth, L. Bo, R. Chu, M. Kung, L. LeGrand, J. R. Smith, and D. Fox, "Gambit: An autonomous chess-playing robotic system," in *2011 IEEE International Conference on Robotics and Automation*, May 2011, pp. 4291–4297.

[3] T. D. Phuc and B. C. Son, "Development of an autonomous chess robot system using computer vision and deep learning," *Results in Engineering*, vol. 25, p. 104091, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590123025001793

[4] S. Bennett and J. Lasenby, "Chess - quick and robust detection of chess-board features," 2013. [Online]. Available: https://arxiv.org/abs/1301.5491

[5] G. Wölflein and O. Arandjelović, "Determining chess game state from an image," *Journal of Imaging*, vol. 7, no. 6, p. 94, Jun. 2021. [Online]. Available: http://dx.doi.org/10.3390/jimaging7060094

[6] A. T.-Y. Chen and K. I.-K. Wang, "Robust computer vision chess analysis and interaction with a humanoid robot," *Computers*, vol. 8, no. 1, 2019. [Online]. Available: https://www.mdpi.com/2073-431X/8/1/14

[7] S. Kluziak and P. Kohut, "Development of a ur5 cobot vision system with mlp neural network for object classification and sorting," *Information*, vol. 16, no. 7, 2025. [Online]. Available: https://www.mdpi.com/2078-2489/16/7/550

[8] L. Kobras, B. Meussen, and M. Soll, "Didactic design of a remote collaborative robotics laboratory," in *2023 IEEE 2nd German Education Conference (GECon)*, Aug 2023, pp. 1–6.

[9] M. A. Goodrich and A. C. Schultz, "Human–robot interaction: A survey," *Foundations and Trends in Human-Computer Interaction*, vol. 1, no. 3, pp. 203–275, 2007. [Online]. Available: https://doi.org/10.1561/1100000005

[10] J. Wainer, D. J. Feil-Seifer, D. A. Shell, and M. J. Matarić, "The role of physical embodiment in human–robot interaction," in *15th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*, Sep. 2006, pp. 117–122. [Online]. Available: https://doi.org/10.1109/ROMAN.2006.314404

[11] A. Pereira, C. Martinho, I. Leite, and A. Paiva, "Icat, the chess player: The influence of embodiment in the enjoyment of a game," in *7th International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS 2008*, ser. Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS. International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), 2008, pp. 1229–1232.

[12] S. Honig and T. Oron-Gilad, "Understanding and resolving failures in human–robot interaction: Literature review and model development," *Frontiers in Psychology*, vol. 9, p. 861, 2018. [Online]. Available: https://doi.org/10.3389/fpsyg.2018.00861

[13] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000. [Online]. Available: https://doi.org/10.1109/34.888718

[14] A. Duda and U. Frese, "Accurate detection and localization of checkerboard corners for calibration," in *Proceedings of the 29th British Machine Vision Conference*. BMVA Press, 2018, p. 126. [Online]. Available: https://bmvc2018.org/contents/papers/0508.pdf

[15] G. Fabris, L. Scalera, and A. Gasparetto, "Playing checkers with an intelligent and collaborative robotic system," *Robotics*, vol. 13, no. 1, 2024. [Online]. Available: https://www.mdpi.com/2218-6581/13/1/4

[16] G. Hoffman, "Evaluating fluency in human–robot collaboration," *IEEE Transactions on Human-Machine Systems*, vol. 49, no. 3, pp. 209–218, Jun. 2019. [Online]. Available: https://doi.org/10.1109/THMS.2019.2904558

[17] P. A. Lasota, T. Fong, and J. A. Shah, "A survey of methods for safe human–robot interaction," *Foundations and Trends in Robotics*, vol. 5, no. 4, pp. 261–349, 2017. [Online]. Available: https://doi.org/10.1561/2300000052